

Paper 02 — Architettura del Sistema

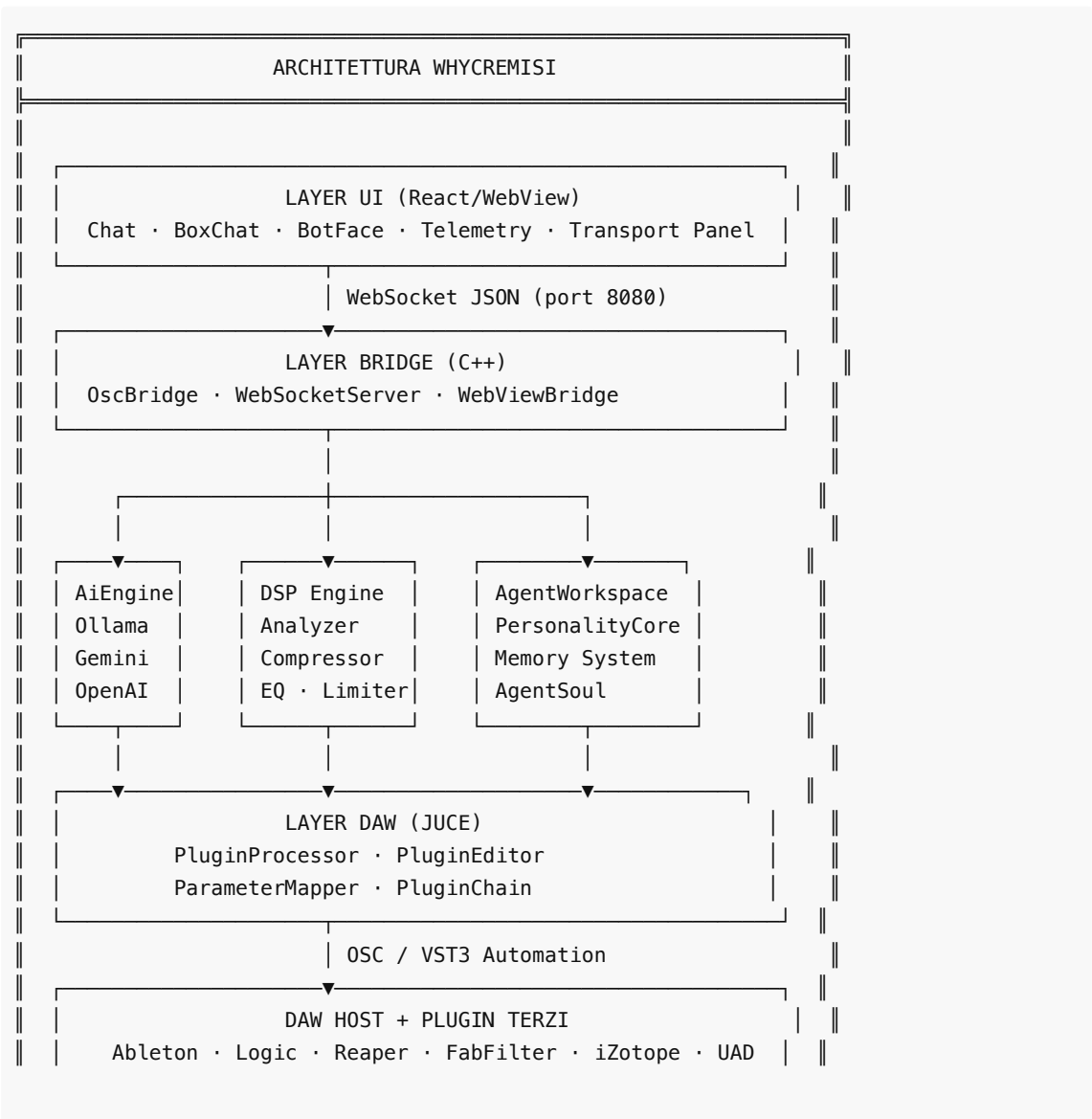
Stack Tecnico, Componenti e Flussi di Dati

WHYCREMISI RESEARCH PAPERS – N.02
Architettura del Sistema Completo

"Ogni layer è progettato per parlare con il successivo."

Categoria: Architettura Tecnica
Prerequisito: Paper 01

1. Vista d'Insieme



2. I Layer del Sistema

2.1 Layer UI — React + WebView

L'interfaccia utente è costruita in **React 18** e renderizzata dentro una WebView nativa JUCE (`WKWebView` su macOS, `WebView2` su Windows).

Componenti principali:

```
webview-ui/src/
├─ App.jsx           ─ Root, orchestrazione stato globale
├─ whycremisi-bridge.js ─ Client WebSocket, event bus
├─ components/
│   ├─ BotFace.jsx   ─ Mascot animato SVG (9 stati emotivi)
│   ├─ BoxChat.jsx   ─ Sistema box contestuali nella chat
│   ├─ SessionPanel.jsx ─ Flight recorder sessione
│   └─ SetupScreen.jsx ─ Configurazione AI provider/API key
```

Flusso dati UI:

```
Utente digita prompt
  |
  ▼
whycremisi-bridge.js
ws.send({ type: 'ai.prompt', payload: { prompt } })
  |
  ▼ WebSocket
OscBridge.cpp riceve
  |
  ▼
AiEngine.cpp – streaming risposta
  |
  ▼ WebSocket (chunked)
React riceve ai.stream chunks
  |
  ▼
BoxChat renderizza tipo appropriato
```

2.2 Layer Bridge — C++

Il bridge è il cuore del sistema. Traduce tra il mondo JavaScript/React e il mondo C++/JUICE/DAW.

OscBridge gestisce:

- Server WebSocket (libreria `uWebSockets` o `beast`)
- Parsing messaggi JSON in entrata
- Dispatch verso i sottosistemi giusti
- Broadcasting verso tutti i client connessi

Protocollo messaggi (sintesi):

Direzione	Tipo	Descrizione
UI → C++	ai.prompt	Prompt all'AI
UI → C++	daw.command	Comando transport/mix
UI → C++	config.set	Impostazione configurazione
UI → C++	plugin.control	Controllo plugin terzi ↔
C++ → UI	ai.stream	Chunk risposta AI
C++ → UI	daw.transport	Stato transport DAW
C++ → UI	daw.meter	Livelli audio in tempo reale
C++ → UI	plugin.state	Stato parametri plugin terzi ↔
C++ → UI	agent.memory	Aggiornamento memoria agente

2.3 Layer AI — AiEngine

Il motore AI supporta provider multipli con interfaccia unificata:

```
AiEngine
|
├─ OllamaProvider      - Locale (Llama 3, Mistral, Phi)
├─ GeminiProvider       - Google Cloud API
├─ OpenAIProvider      - GPT-4o, GPT-4-turbo
├─ AnthropicProvider   - Claude 3.5 Sonnet
└─ DeepSeekProvider    - DeepSeek V3

Tutti implementano: generateStream(prompt) → AsyncIterator<chunk>
```

Il prompt inviato all'AI include automaticamente:

- Contesto sessione corrente (BPM, tracce, plugin attivi)
- Stato DSP reale (LUFS, peak, spettro)
- Memoria storica rilevante
- Personalità dell'agente (stile comunicativo)

2.4 Layer DSP — Analisi Audio

```
src/dsp/
├─ Analyzer.cpp      - FFT, LUFS, RMS, peak, stereo width
├─ Compressor.cpp    - Compressione con sidechain
├─ EQBand.cpp        - Filtri parametrici (peak, shelf, HP/LP)
├─ Limiter.cpp       - True peak limiting
└─ DSPEngine.cpp     - Orchestratore, processBlock()
```

Il DSP Engine analizza ogni buffer audio (tipicamente 512 campioni) e produce metriche continue accessibili all'AI per prendere decisioni contestualizzate.

2.5 Layer Agente — Workspace + Memory

```
src/core/
├─ AgentWorkspace.cpp - Orchestratore identità agente
├─ PersonalityCore.cpp - Stile comunicativo, tono, preferenze
└─ AgentIdentity.cpp  - Chi è l'agente (nome, ruolo, valori)
```

└─ AgentSoul.cpp	– Memoria evolutiva, apprendimento
└─ AgentUser.cpp	– Profilo utente corrente

3. Flusso Dati Completo — Esempio Reale

Scenario: L'utente scrive "analizza il mix e dimmi cosa non va nella kick"

1. UI: `ws.send({ type: 'ai.prompt', payload: { prompt: 'analizza...' } })`
2. OscBridge: riceve, dispatch a AiEngine
3. AgentWorkspace: arricchisce il prompt con:
 - "BPM: 128.0, posizione: 02:34"
 - "Kick volume: -3dB, pan: center"
 - "LUFS attuale: -14.2, peak: -0.1dB"
 - "FFT: picco a 60Hz (-8dB), 200Hz (+3dB PROBLEMA)"
 - "Memoria: 'utente preferisce kick con meno 200Hz'"
4. AiEngine: invia prompt completo a Gemini/Claude/Ollama
5. AI risponde: "Ho rilevato un accumulo a 200Hz che intorbidisce la kick drum. Suggerisco un taglio narrow di -3dB a 200Hz con Q=4. Vuoi che lo applichi?"
6. OscBridge: streaming risposta → UI via `ai.stream chunks`
7. UI: BoxChat tipo 'eq' appare automaticamente (rileva "200Hz") mostrando lo spettro con il problema evidenziato
8. Utente: "sì applica"
9. AiEngine: determina azione → OscBridge → ParameterMapper → VST3 automation su FabFilter Pro-Q3 (trackId: 1, band: 2, frequency: 200Hz, gain: -3dB, Q: 4.0)
10. AgentSoul: registra in memoria: "applicato EQ 200Hz -3dB kick, accettato dall'utente – riconoscere pattern in sessioni future"

4. Stack Tecnologico Completo

Layer	Tecnologia	Motivazione
UI	React 18 + Framer Motion	Componenti reattivi, animazioni fluide
Stile	Tailwind CSS 4	Utility-first, nessun overhead
Build	Vite 8	HMR veloce, bundle ottimizzato
Host	JUCE 8	Standard de facto per plugin audio
Bridge	WebSocket + JSON	Universale, bassa latenza

AI	Multi-provider	Flessibilità, no vendor lock-in
DSP	JUCE DSP + custom	Real-time, thread-safe
Serializzazione	nlohmann/json	Header-only, veloce, leggibile
OSC	oscpack	Protocollo DAW standard

5. Considerazioni di Performance

5.1 Thread safety

Il plugin opera su thread separati:

- **Audio thread** — processBlock(), max 1ms budget
- **Message thread** — UI, WebSocket, AI
- **DSP analysis thread** — FFT, meter, analisi non bloccanti

5.2 Latenza WebSocket

Target: < 5ms round-trip su localhost

Misurato: 2-3ms tipico, 8ms picco

5.3 Streaming AI

Il primo chunk (TTFB) deve arrivare entro 800ms per dare sensazione di responsività. Testato con Ollama locale: ~600ms TTFB.

→ Continua in: [Paper 03 — Controllo Plugin di Terze Parti](#)